ARCHITECTURE

Foxtrot DLL architecture

Projects and ownership

The solution separates the former Core implementation from the foundational Core DLL. Existing runtime classes retain their `Core::` namespace to limit source churn; their implementation and `CORE_API` exports now belong to **EngineRuntime.dll**. `FT_CORE_API` is the distinct export macro for the foundational **Core.dll**.

```
FoxtrotEngine_DirectX.sln
Engine/
  Main/           EXE bootstrap and module host
  Core/           DLL: logging, time, allocator, module ABI, shared file serialization
  EngineRuntime/  DLL: loop, platform windows/input, actors, scenes, CPU assets
  D3D11/          DLL: Direct3D device, presentation, render components, GPU assets
  Game/           DLL: game-owned state and registered update systems
  Editor/         optional DLL: ImGui tools and editor-specific interaction
  Math/           static library: math helpers
  GenericData/    static library: CSV, JSON and text resource implementations
  Graphics/       header utility project: existing graphics contracts
  InputSystem/    header utility project: existing input contracts
```

Main produces `Game.exe` in Debug/Release and `FoxtrotEditor.exe` in Foxtrot_Editor_Debug. D3D11 produces **FoxtrotD3D11.dll**, avoiding a filename/import library collision with Windows' `d3d11.dll` / `d3d11.lib` .
Editor is excluded from normal Debug/Release solution builds.

Input stays in Runtime: its Windows polling and the shared message pump do not justify an independent DLL. The old `D3D11InputDevice` name and graphics factory entry remain as compatibility surfaces. Scenes and actor/component infrastructure also remain in Runtime. No Audio or Physics DLL is introduced for unimplemented subsystems.

Dependencies

Arrows below mean binary imports (consumer -> provider):

```
EngineRuntime.dll -> Core.dll
FoxtrotD3D11.dll  -> EngineRuntime.dll, Core.dll
Editor.dll       -> FoxtrotD3D11.dll, EngineRuntime.dll, Core.dll
Game.dll         -> Core.dll
Game.exe / FoxtrotEditor.exe -> no engine DLL imports
```

Game also compiles against Runtime's public `GameServices` header and receives that table at runtime. Runtime calls registered Game callbacks; it does not import Game.dll. The legacy plugin interface similarly lets Runtime invoke D3D11/Editor without an upward import. Math and GenericData are linked into their consumers as static libraries. Main's project references enforce build order only: `LinkLibraryDependencies=false` .

Runtime loading/ownership arrows (the EXE owns all handles):

```
Main::ModuleHost --LoadLibraryExW--> Core.dll
                 --LoadLibraryExW--> EngineRuntime.dll
                 --LoadLibraryExW--> FoxtrotD3D11.dll
                 --LoadLibraryExW--> Game.dll
                 --LoadLibraryExW--> Editor.dll (editor configuration)
```

Core must never import Runtime, graphics, editor, input or game implementations. Runtime must never link FoxtrotD3D11.lib, Editor.lib or Game.lib. D3D11 must never link Editor.lib or Game.lib. Game must not include concrete renderer/editor types. Editor deliberately knows the backend for tooling; this is a compatibility boundary, not a backend-independent editor API.

Core

Public bootstrap headers live under `Engine/Core/include/Foxtrot/Core` and are exposed through the shared MSBuild properties. Building Core produces Core.dll and Core.lib. Every engine DLL links Core.lib and uses Core's exports. The EXE does not link Core.lib: it explicitly loads Core first and resolves `FtGetModuleAPI`.

Core currently provides `FtLog`, `FtSeconds`, `FtAllocate`, `FtDeallocate`, and the module contract. The existing `Common::FileIOHelper` implementation is located at `Engine/Core/src/FileSystem/FileIOHelper.cpp` to share serialization state; it is a same-toolchain C++ compatibility API, not a portable ABI. Math remains a static library of value-level utilities. Foundational path/configuration/error helpers can be added here when actually shared. Core owns no window, graphics device, scene, game loop or game state.

Core's explicit handle remains held until every dependent DLL has shut down, destroyed its state and released its own handle. Normal import references from the other DLLs are additional Windows loader references. `FreeLibrary(Core)` must never be attempted as a way to unload Core while consumers are active.

Bootstrap and lifetime

`Main/src/ModuleHost.cpp` resolves each module beside the executable using an absolute path and `LOAD_LIBRARY_SEARCH_DLL_LOAD_DIR | LOAD_LIBRARY_SEARCH_DEFAULT_DIRS`. It owns handles and stable API records, looks up the undecorated `FtGetModuleAPI` symbol, validates descriptors, then calls initialize. Each DLL has a `.def` file so this symbol is stable on both Win32 and x64.

Validation includes API version, descriptor size, build flavor, pointer width, module name and required function pointers. The factory returns an opaque instance and initialize/shutdown/destroy/query functions. Allocation and destruction of the instance happen inside its producer. Initialization exceptions become status codes; the EXE unwinds failed loads and previously initialized modules. Shutdown must tolerate partial initialization and repeated calls. Heavy work is kept out of DllMain, outside the Windows loader lock.

The EXE's host service performs module lookup among initialized modules. Returned tables are borrowed and valid only while their provider remains loaded. Registry entries do not own plugins. Only ModuleHost loads/unloads engine DLLs; the legacy PluginManager is a deterministic, borrowed plugin registry.

Startup and initialization order:

1. EXE loads and initializes Core.
2. EXE loads Runtime. Runtime initializes COM, its managers and loop state.
3. EXE loads D3D11 and attaches its plugin to Runtime. D3D11 creates its renderer; native window creation is delegated to Runtime's platform API.
4. EXE loads Game. Game queries GameServices and registers its update callback.
5. Editor builds load/attach Editor, initialize its GUI and borrow backend services.
6. Runtime runs the only Windows message pump and game loop. It dispatches input, game callbacks and plugin updates/rendering. Editor owns presentation scheduling in the editor configuration; D3D11 performs GPU work.

Shutdown order, including exception unwinding:

1. Return from the loop and stop the world while all plugin code is loaded. Destroy scene actors/components and cached prefab prototypes before their implementation DLLs can disappear. Plain CPU data stays alive through graphics teardown.
2. Shutdown/destroy/unload Editor when present. Detach native callbacks, release editor objects and GUI state. Borrowed windows/device/input/game camera survive.
3. Shutdown/destroy/unload Game. Unregister callbacks before destroying Game state.
4. Shutdown/destroy/unload D3D11. Release GPU resources, backend singletons, presentation wrappers, camera, factory-owned input objects and device/context.
5. Shutdown/destroy/unload Runtime. Release CPU assets/managers/registry, remaining native windows and registered window class, then balance COM initialization.
6. Shutdown/destroy/unload Core last; exit the EXE.

No callbacks, worker threads, queued functions, virtual objects, COM objects owned by plugin code, or borrowed interfaces may survive their provider's unload. Current registration and lifecycle calls are main-thread-only. Calling registration or removal during callback iteration returns Busy; unload occurs after iteration.

ABI rules

The new bootstrap and Game service boundary use C-linkage exports, explicitly `__cdecl` function pointers, fixed-width integers, opaque context pointers, sizes and status codes. Headers are C++17 headers (including `noexcept` and namespaces), not a C-language SDK. C linkage avoids export-name mangling; it does not by itself make all layouts portable. Use the supplied headers and default structure packing.

All binaries must use the same MSVC toolset, architecture, configuration and CRT: v145, `/MDd` in Debug/editor and `/MD` in Release. Do not mix Debug, Release or editor DLLs, different iterator-debug settings, packing options, or independently compiled STL implementations. BuildAbi rejects the three supported build flavors; it is not a complete compiler fingerprint.

New public services should follow these rules:

- Keep STL containers, streams, exceptions, RTTI and owning C++ classes inside a module. Use handles or pointer/length views with explicit borrowed lifetimes.
- Allocate/destroy objects in the same DLL. For shared raw buffers, pair Core's allocator/deallocator and keep Core loaded through deallocation.
- Catch exceptions before returning across the service boundary; return Status.
- Version tables when changing layout/signatures. Callbacks must obey `FT_CALL` and `noexcept`; no retained callback may point into an unloaded module.
- Keep DirectX resources in D3D11. Existing Editor COM access is borrowed unless explicitly AddRef'd; release every retained interface before backend teardown.

Existing renderer/component/editor interfaces still use exported C++ classes, virtual functions, custom containers, STL streams and some cross-module object operations. This migration preserves that **same-toolchain compatibility layer**; it does not claim those old interfaces are a stable third-party plugin SDK. The C4251/C4275 warnings expose that remaining coupling. Replacing it with versioned resource/entity handles is a later migration, not accomplished by the C entry point.

Game DLL

`Engine/Game/src/GameModule.cpp` owns GameState and registers a system with the Runtime GameServices table. Its producer destroys its state; Runtime only borrows the callback/context. The sample system demonstrates registration and execution; the existing solution did not provide a complete separate game to port.

Add gameplay rules, orchestration and game-specific systems here. Extend Runtime's service table with versioned entity/resource operations when gameplay needs them, rather than importing concrete D3D11 implementations. Component registration is not

yet a public GameServices operation; the old internal component factories remain available to the compatibility layer.

`--reload-test` proves full teardown followed by a fresh session in the same EXE. It is not live hot reload. Future live replacement needs callback/job quiescence, destruction of all game-defined objects, and host-owned serialized state with a versioned restore contract.

Build and validation

Install Visual Studio Desktop development with C++, v145, Windows SDK 10 and Git. Debug/editor graphics creation also requires the Windows D3D11 debug layer and a feature-level-11-capable graphics device. `setup.bat` invokes the pinned dependency setup script; existing dependency checkouts are preserved and checked by revision. The tested third-party revisions are in `script/setup-dependencies.ps1`. The missing stb image-write header is vendored with its license in D3D11/vendor.

From the solution root:

```
./script/setup-dependencies.ps1
./script/build.ps1 -Configuration Debug -Platform x64
./script/test-modules.ps1 -Configuration Debug -Platform x64
```

Repeat with Release and Foxtrot_Editor_Debug, and with x86. Solution x86 maps to project Win32. `./script/verify-solution.ps1` builds and tests all six combinations. Binaries are isolated in `build/<x64|Win32>/<configuration>/` with per-project intermediate directories. Keep all DLLs from the same output folder together. Set Main as the Visual Studio startup project; normal run is unbounded until window close.

Tests launch the built EXE from an isolated test-data directory. They verify two three-frame sessions, execution of both Game callbacks, all engine handles gone after teardown, rollback after an injected failure immediately after Runtime initialization, and rollback when Game.dll is missing after D3D11 initialization. The missing-DLL case uses a separate copy of the outputs. Logs are beside the executable. These are startup/render-loop and DLL-lifecycle smoke tests, not full asset, scene editing, input or gameplay tests. Existing asset paths/workflows still require project content and separate functional testing. The migration does not promise warning-free legacy code.

Verified migration result (2026-09-07)

`script/verify-solution.ps1` completed successfully with MSVC 14.51.36231/v145: Debug, Release and Foxtrot_Editor_Debug each built on x64 and Win32. All 18 lifecycle cases passed against the final sources. The aggregate local log is `build/verification.log`; per-configuration build logs and test traces remain under `build/`. Binary import inspection confirmed that both EXEs have no engine imports, Core has no upward engine imports, and all four consuming DLLs import Core. `git diff --check` also passed. That validation preceded the compiler warning cleanup described below; full content/editor workflows were outside its scope.

Cursor and presentation coordinates

Native window back buffers and presentation viewports use the actual Win32 client size, including after resize. The Scene image has its own render-texture size; resizing or docking that image must not resize the editor's presentation surface. Only the editor HWND forwards input to its ImGui backend. The separate Game HWND retains the close check but does not feed its client-relative mouse positions into the editor's UI context.

Input mouse positions are signed client coordinates, including negative values outside the window. `(0,0)` is a valid position, and wheel-message screen coordinates do not replace the client coordinates. Devices poll once per frame using the focused engine window. Scene hover is determined by the displayed image, not window focus.

`script/test-cursor.ps1` builds the standalone `tests/CursorRegression.vcxproj` and tests negative/origin/wheel coordinates plus real D3D11 back-buffer and viewport alignment before and after resizing a test-owned native window. The full `verify-solution.ps1` workflow also runs this regression for every configuration.

The cursor revision passed all six builds and all 24 lifecycle/cursor cases. Results are in `build/cursor-verification.log`, with the final Debug x64 offscreen viewport refresh recorded in `build/cursor-debug-refresh.log`.

Physical ownership of formerly shared files

The former Engine/Common files now reside in their owning projects. Core holds shared containers, debugging/file helpers and foundational resource types; EngineRuntime holds actor/component/plugin contracts, ResourcePack and their runtime implementations; Editor holds EditorHelper. Existing Common:: namespaces remain unchanged. Project includes and source items use the new locations, and the unused Common.vcxitems manifest has been retired. Dormant sources remain visible as non-compiling project items. No build or tests were run during the relocation itself.

Compiler warning cleanup

Legacy classes containing STL containers, math values or COM smart pointers export their out-of-line methods individually instead of exporting the entire class. SceneManager and DebugShapes keep their singleton storage and creation/destruction functions in their owning DLL, so clients cannot create separate singleton copies. These legacy APIs still require the same MSVC toolset, configuration and CRT across modules; this cleanup does not make their C++ layouts a portable ABI.

ResourcePack destruction requires a complete resource type, and ResourceManager includes the JSON/text definitions before instantiating deletion. Renderer components explicitly resolve their shared IComponent virtual methods. Index and pixel-size conversions are checked or made explicit, and decoded image memory is released with stbi_image_free through RAII.

Compiler warning C4244 is disabled only for the pinned upstream Spine runtime's source files. All engine sources retain their existing warning level and conversion diagnostics; DLL-interface and incomplete-deletion diagnostics are not suppressed.

After this cleanup, MSBuild Rebuild completed with zero warnings and zero errors for Debug, Release and Foxtrot_Editor_Debug on both x64 and x86. Results are recorded in build/warning-clean-results.txt, with per-configuration logs named build/warning-clean--.log. The compiler command records also confirm that the Spine suppression does not apply to engine sources. No tests or application executables were run for this cleanup.